

Discrete Population Models

Nicole DePergola

February 2024

1 Introduction

Markovian and Stochastic approaches were experimented with in this lab. In order to model table occupancy probabilities for a restaurant, a Markovian process is powerful in that each step only depends on the previous step. Therefore, only know the arrival and departure times for the restaurant, we can see the probabilities change over an infinite amount of time. Improving this model, we can have a Stochastic approach. This introduces randomness that more accurately represents reality. In the context of a city, people make random movements and have unique characteristics in terms of susceptibility to illnesses and recovery times. With randomness, complex systems can be analyzed and more comparable to real-world scenarios where uncertainty is inevitable.

2 Methods & Results

2.1 Restaurant

We can model customers occupying tables in a restaurant using the Markovian Process. Figure 1 shows a visual of the probabilities to get from one state to another. There are only two rates involved in this system: the arrival rate p and the departing rate q . Table 0 is defined as no tables occupied. There are two routes to go from here: either no one enters the restaurant (probability $1 - p$) or customers occupy one table (probability p).

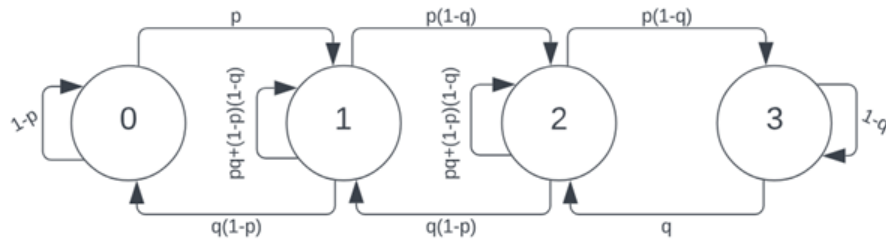


Figure 1: State transition diagram for tables being filling in a restaurant.

While one table in the restaurant is occupied, there are three outcomes that can happen next. First, more customers could come and fill a second table in the restaurant. In order for

this to happen, the customers have to arrive at rate p , at the same time that the customers at table 1 are not leaving, rate $1 - q$, so we're left with the probability $p(1 - q)$. Second, $pq + (1 - p)(1 - q)$ corresponds to the probability of only one table still being occupied after the time step. A party can arrive at the same time a party is leaving that table (pq term), or no one can leave and no one can arrive during the time step (the $(1 - p)(1 - q)$ term). Lastly, the restaurant can experience one table being occupied, to no tables being occupied. This is the $q(1 - p)$ probability on Figure 1.

State changes from 1 to 2 remain the same. State 3, the last option, defined as the probability of 3 tables being occupied at any time, is slightly different. After a time step, either three tables are still occupied (no one leaves so probability $1 - q$) or the party leaves with probability q and now only two tables are occupied.

We can arrange the probabilities from Figure 1 into a matrix, shown here:

$$M = \begin{bmatrix} 1 - p & q(1 - p) & 0 & 0 \\ p & pq + (1 - p)(1 - q) & q(1 - p) & 0 \\ 0 & p(1 - q) & pq + (1 - p)(1 - q) & q \\ 0 & 0 & p(1 - q) & 1 - q \end{bmatrix}$$

The first column represents the probabilities of moving out of state 0, no tables occupied. The second column has the probabilities associated with one table being occupied, and so on.

A useful property of Markovian matrices and processes is the following relationship:

$$x_{\infty} = M^{\infty} x_0 \tag{1}$$

where x_0 is called the initial state vector,

$$x_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

so 100% of customers are not at a table to start. Over time, following Equation 1, the state vector will change as it is multiplied by the transition matrix M . The next time step in a Markovian process only depends on the previous step.

To carry out this system, I defined the following rates for p and q :

$$p = 0.3 \quad q = 0.2$$

and created a loop that will perform Equation 1 for 100 time steps. I plotted the probability of each table being occupied over time in Figure 2.

We see that the probability of one table being occupied rapidly increases when the restaurant opens. The probabilities level out to their steady state values. This means that this probability will continue indefinitely. We can use these steady state probabilities as a tool to figure out the best number of tables this restaurant would need to hold the maximum number of people that want to eat there. Based on our rates for p and q , we see that the greatest probability (39.27%) is for all tables being occupied at a given time. The restaurant is most likely always at full capacity. This tells us that we should get more tables for this

restaurant so it can hold more customers at once, since there is clearly a demand for more tables.

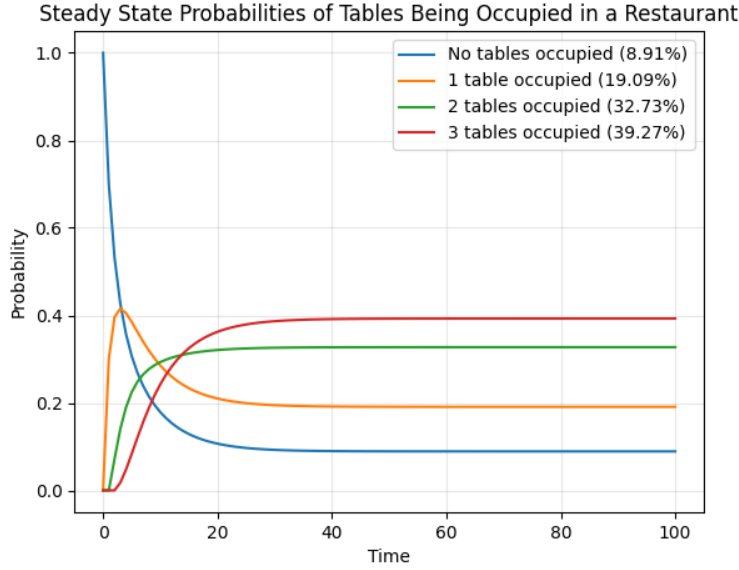


Figure 2: Probability of tables being occupied in a restaurant over time.

Table 1 shows the occupancy probabilities at various times through the simulation from Figure 2. As the state vector progresses, the probabilities converge to their steady state values.

Table 1: Probabilities of tables being occupied in a restaurant.

Probabilities of Table Occupancy Over Time				
State Vector	No tables	1 table	2 tables	3 tables
x_0	1	0	0	0
x_1	0.7	0.3	0	0
x_2	0.532	0.396	0.072	0
x_3	0.42784	0.4152	0.13968	0.01728
x_4	0.357616	0.4053312	0.1897056	0.0473472
x_5	0.30707757	0.38514893	0.2243664	0.0834071
...				
x_{15}	0.13569197	0.24094685	0.31017279	0.31318838
...				
x_{20}	0.10984526	0.21327654	0.31971061	0.35716758
...				
x_{100}	0.08909095	0.19090914	0.32727271	0.3927272

2.2 City

To begin building a city, I started by defining a grid of size 50x50 with wrap-around borders using a Class. This Pac-Man style layout is easy to achieve by using modulo; if a position is reached outside the constraints of the grid, modulo will bring it back inside.

My next task was to have one person start walking around this ‘city’. I created a Person Class that holds the position of the person and functions on how they can move. They are placed at a random position on the grid using a random integer generator with bounds the size of the grid. I defined the movement function to be separated into x-direction and y-direction. A random number is chosen where -1 indicates a negative x-step, 0 is no movement, and 1 is a positive x-step. The same goes for the y-direction. So, people can move up, down, left, right, diagonally in any direction, or not move at all. And if they stepped off the grid, a modulo will take care of this and correct their position to the opposite side of the grid.

Next, I implemented some obstacles. In my Grid Class, I defined a function that allows me to set cells equal to a value. I created a random number generator that will set a fixed number of cells on the grid equal to 1. I then had to update my Person Class to respond effectively. If the person wants to update their position to a cell that is equal to 1, they cannot do so and have to try another move. See this simple system in Animation #1 (in email). The obstacles are visualized as the black blocks.

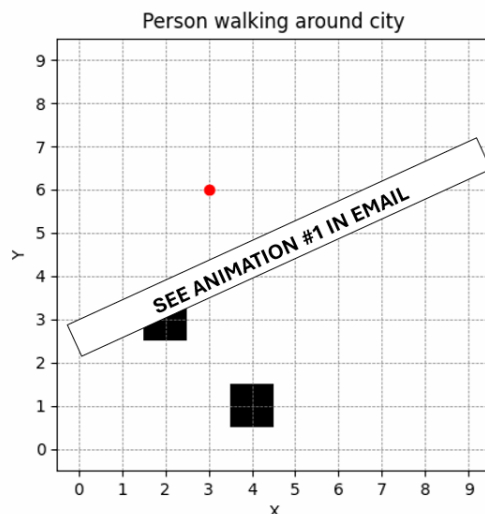


Figure 3: Person walking around city with obstacles.

From here, I introduced more people by calling the Person Class several times to make the desired number of unique people, and ensured they walked around correctly on my grid, avoiding all randomly placed obstacles. Now is the time to introduce an illness in the population. I created my population of 420 people, and labeled the first person as sick. I defined another function in my Person Class to cause the spread of infection. As a loop runs through to update every persons position, this function will detect if a sick person is on the same cell as a healthy person. If they are on the same space, the healthy person has a 95% chance of becoming sick. A random number generator determines if they will become sick or not according to this probability, and then they are also labeled as sick.

Another function in the Person Class will then update how long each sick person has been sick. Sick people are guaranteed to be contagious for 25 time steps. After this, each person has a unique probability of losing the sickness, between 10% and 30%. A random number generator determines if they lose their sick status, and then they reenter the healthy population.

Figure 4 shows a snapshot from the initial state and the final state after 80 steps. This epidemic spread is shown more effectively in Animation #2 (in email), a snapshot from which is seen in Figure 5.

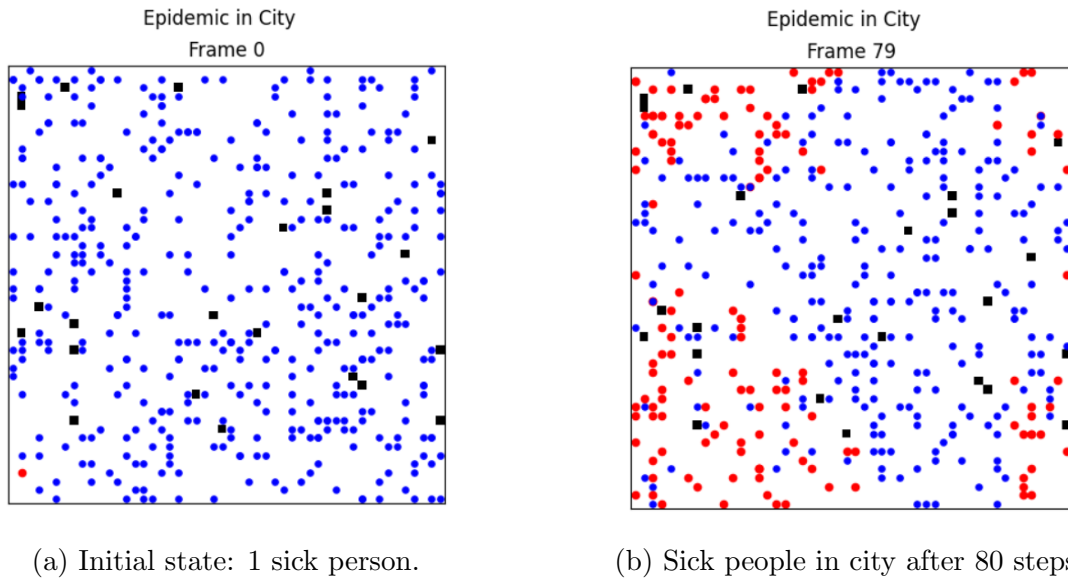


Figure 4: An epidemic spreading through a city of population 420.

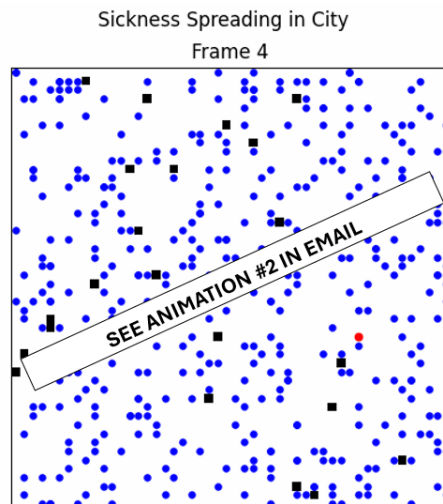


Figure 5: Animation of epidemic spread in city with population 420.

I wanted to see how stable this simulation was, so I ran it 500 times and stored the final number of sick people after each trial. Figure 6 shows the number of times a trial ended with a certain number of sick people. The mean and median both lie around 92 sick people (see Table 2). The standard deviation value of 40.33407 suggests that there is a wide range results that the simulations yield, which is also evident in Figure 6. There appears to be another peak at zero, indicating the epidemic died off before frame 80. I'm not sure why this is in the distribution, there isn't a similar peak on the high end around 180.

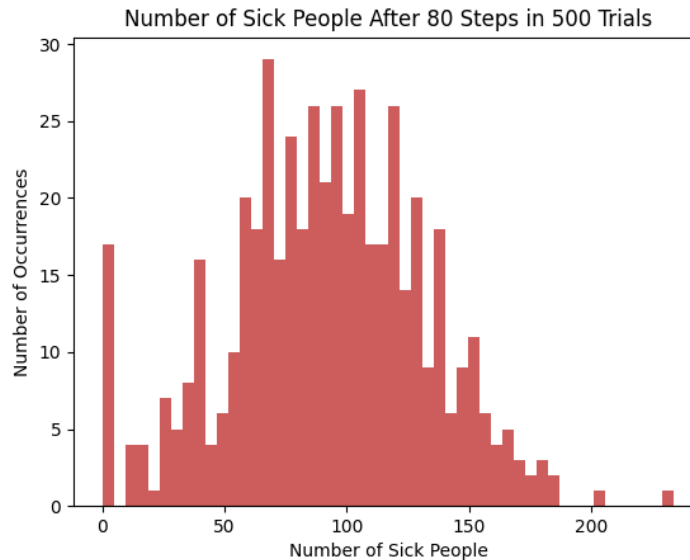


Figure 6: Sick people in city after 80 time steps over 500 trials of epidemic simulation.

Table 2: Statistics of Figure 6.

Mean	Median	Standard Deviation
91.896	92.5	40.33407

3 Discussion

This lab introduced me to Markovian processes and made me very comfortable with classes and animation techniques. The bulk of my time was spent trying to create animations; I fought with my computer for countless hours. I enjoyed the restaurant problem, although I found it hard to implement for larger systems. I'm not sure how to create a larger matrix automatically, but I think that would be the best way to increase the table number in the restaurant and figure out how many tables would be best for the restaurant to have based on the p and q values. I feel it would be counter intuitive to determine the larger matrices by hand – I want my computer to do the work for me! I would like to play around with this more, as the Markovian process is very convenient and informational in such systems.

The city system was quite difficult to get a grasp on and troubleshoot errors. I was fascinated by the wrap-around grid at first, and then the daunting task of adding people ensued. I'm glad I gained more experience with building and using classes. They have proven to be very powerful tools when simulating large amounts of objects that have abilities and characteristics. I would have included ChatGPT as a co-author on this, but I had to troubleshoot so much code and perform so much research during this assignment that I definitely deserve to be the sole author of this work.

I couldn't be happier that I finally got the animations to work. Like I said, that was the biggest headache, by far. As for the histogram I created (Figure 6) I need to investigate why there is a peak at zero. Perhaps this is a representation of how difficult it is to start an epidemic! Or the more likely reason... there's a bug in my code. I'll keep my optimism and believe the former.